

Rust-based Openstack-Keystone

Thinh Nguyen, Kevin Wang

Dominic Murphy, Zebang Li

What is OpenStack?

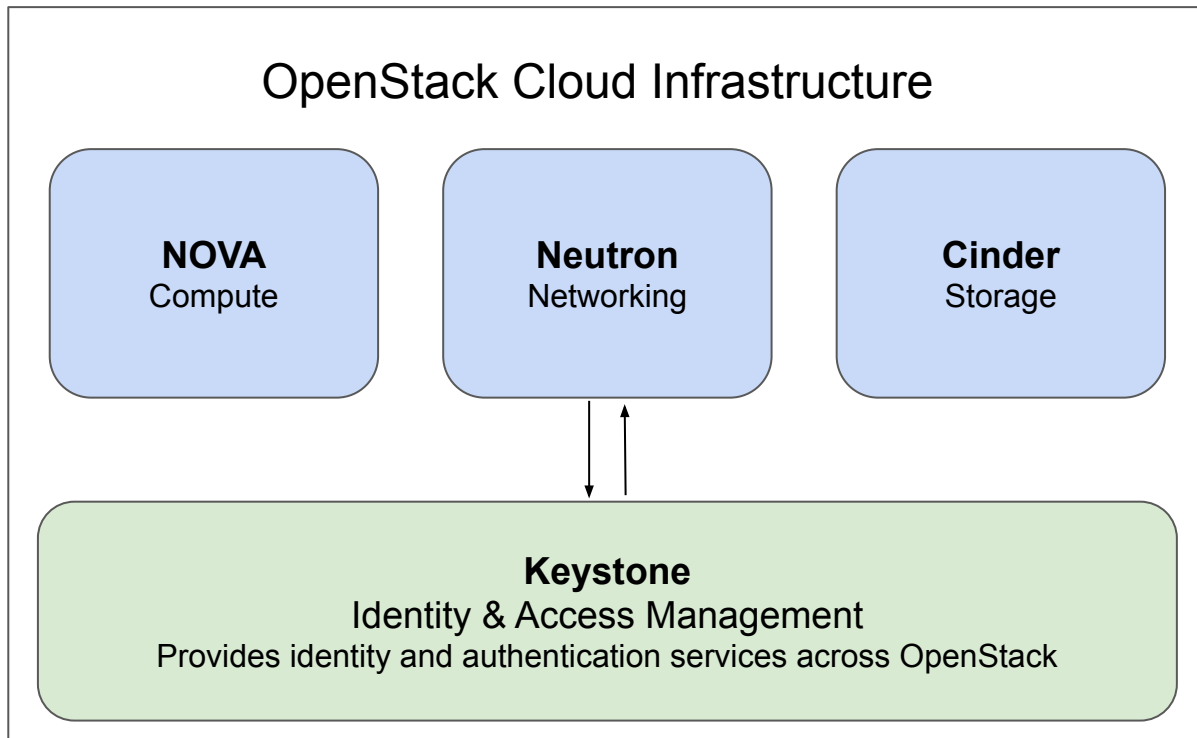
OpenStack:

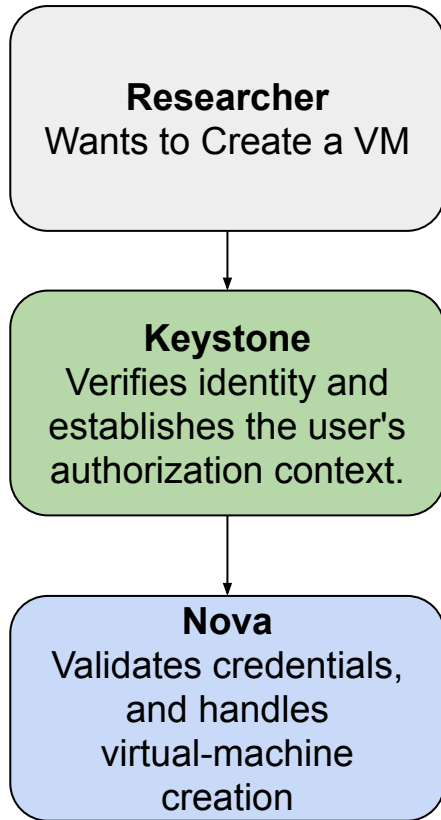
Open-source software for building and managing cloud infrastructure

Keystone:

Provides centralized identity and access management across OpenStack services.

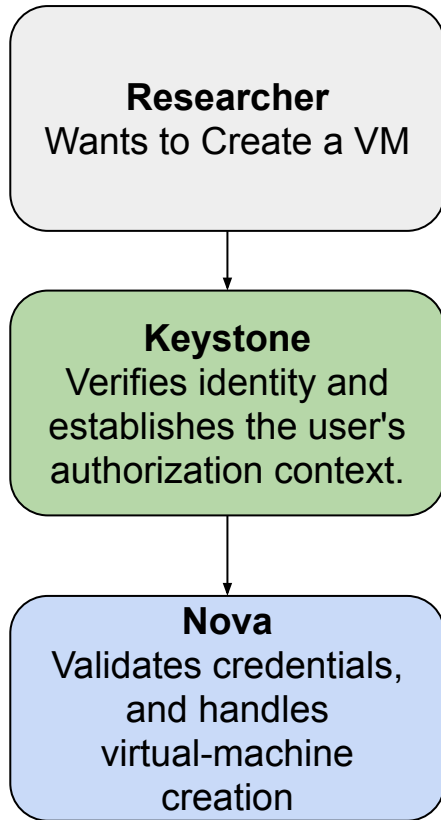
Our focus: Keystone





Supposed BU operates a private cloud with computing resources for research laboratories.

If a researcher needs a virtual machine, they would submit a request through OpenStack.



When the researcher tries to logs in,
3 important “Checks” are made to generate an
“access badge” (*token*).

1. **Authentication** - “Who are you?”
2. **Scope** - “What resources are you accessing?”
3. **Authorization** - “Are you allowed to perform this operation?”

Keystone authorization problem can results in serious problems !
Ex: unauthorized access, incorrect permissions,etc.

Current Keystone Implementation

- Existing Python Keystone is effective and reliable.
- However, lack modern authorization capabilities like WebAuthn, workload authorization, etc.
- In addition, several technical debts such as aging or abandoned dependencies as well as redundant database-access patterns needs to be changed to support modern identity verification.

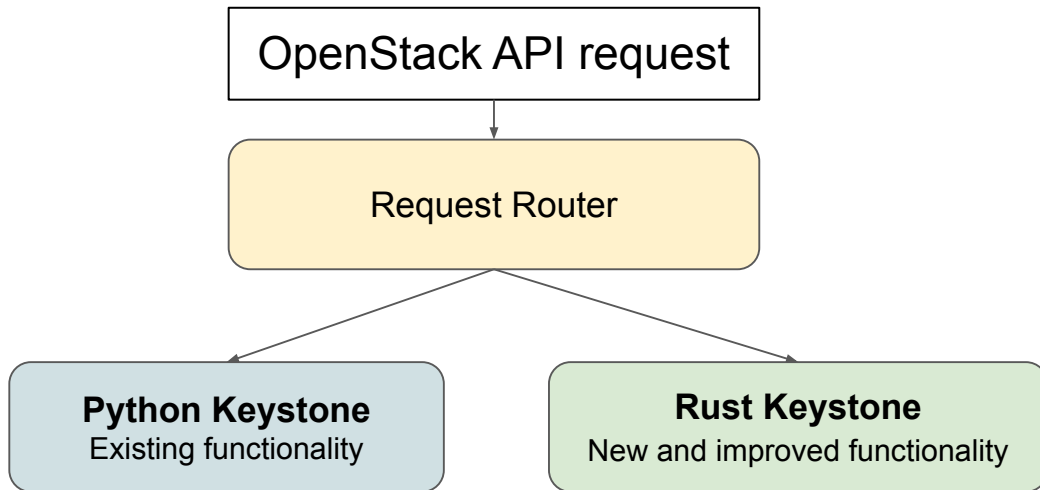
Goal: Introduce new capabilities while maintaining compatibility with existing deployments.

Rust-based Keystone Implementation (Keystone-rs)

Idea:

Creating a Rust Keystone that can work alongside existing Python version.

Modernize the infrastructure incrementally without immediately disrupting existing OpenStack deployments !



Rust-based Keystone Implementation

Rust Advantage:

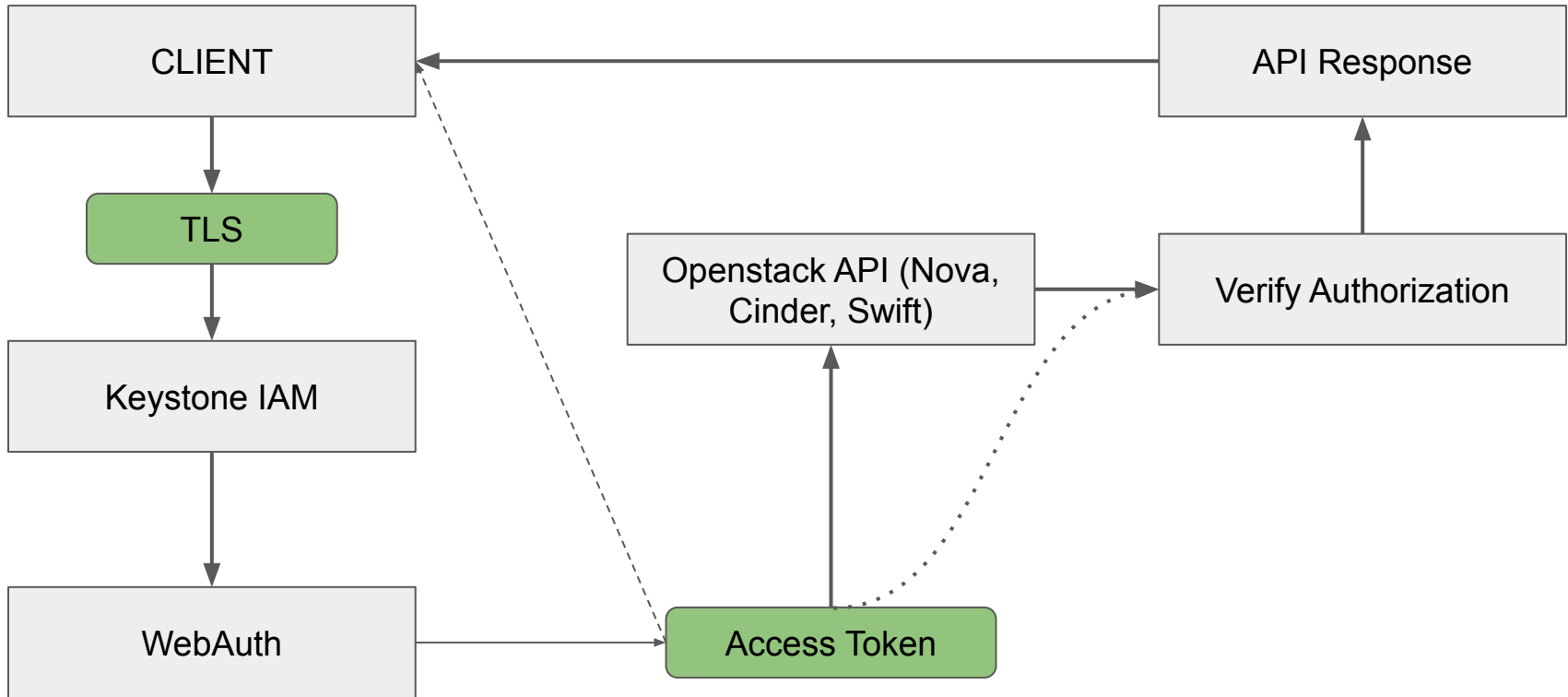
- No interpreter compiling overheads (Compared to Python)
- Supports parallel execution and efficient async concurrency (software side)
- No accumulation of technical debt, allowing for a more better design and modular architecture
- Strong compile-time memory-safety guarantees for safe Rust

Project Design

Keystone has a few designs, we have settled on implementing the following:

- OAuth2 token exchange
 - Allows a client to exchange old authentication tokens with a server
 - Delegation chaining
- Working mTLS+Webauthn (RFC 8705) authentication
 - TLS provides channel provide between client and server
 - Webauthn provides strong client verification

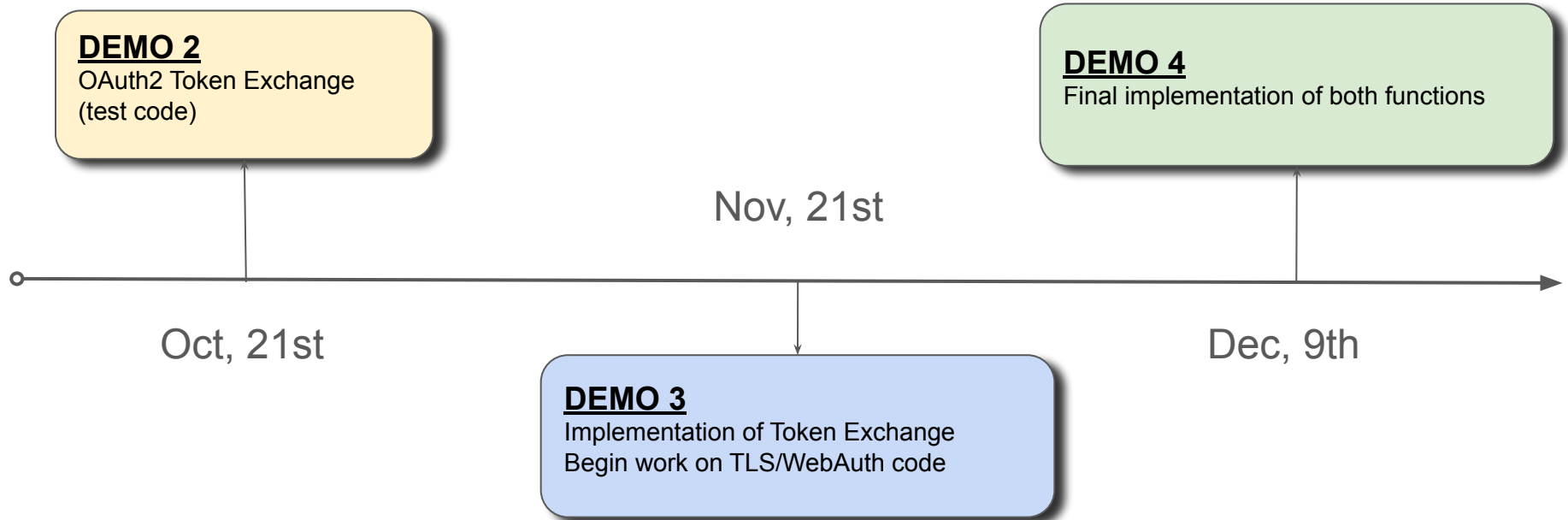
Workflow



Challenges

- Compatible and work across Openstack services
- Code will be used within industry services
- Backwards compatibility with Python
- Security protocols require thorough adherence to standards
 - OAuth 2.0 and OIDC involve security critical operations such as
 - Token issuance and validation
 - Client authentication
 - Cryptographic verification
 - Expiration handling
- **Small errors can have drastic impacts on services and or be exploited!**

Timeline



Thank you! Questions?